Mean-Variance Portfolio Optimization
Piotr Rak 284213

1 Description

Portfolio optimization is the process of selecting asset weights according to an investment objective and a set of constraints. This project focuses on mean-variance portfolio optimization, where the goal is to balance expected return and investment risk.

Historical market data is used to estimate the expected return vector μ and covariance matrix Σ. A ready Kaggle dataset containing S&P 500 company data was used instead of collecting data manually from Yahoo Finance. The dataset covers more than 25 years of prices starting from January 2000, including major events such as the dot-com crash and the 2008 financial crisis. A limitation is survivorship bias, because removed, delisted, or bankrupt companies may not be included.

For asset i, the simple return at time t is:

$$r_{i,t} = \frac{P_{i,t} - P_{i,t-1}}{P_{i,t-1}}$$

The expected return is estimated as the historical average:

$$\mu_i = \frac{1}{T} \sum_{t=1}^{T} r_{i,t}$$

Portfolio expected return and variance are:

$$\mu_p = w^T \mu = \sum_{i=1}^{n} \mu_i w_i, \qquad \sigma_p^2 = w^T \Sigma w$$

where w is the vector of portfolio weights, n is the number of assets, and Σ is the covariance matrix. The covariance matrix is important because total portfolio risk depends not only on individual asset risk but also on how assets move together. Diversification can reduce risk when assets are not perfectly correlated.

The problem is solved in two ways. First, Gurobi is used as an exact quadratic optimization solver. Second, Particle Swarm Optimization is used as a metaheuristic method and compared with Gurobi.

2 Objective and Constraints

The model maximizes expected return while penalizing variance:

$$\max_{w} \left(w^T \mu - \lambda w^T \Sigma w \right)$$

where λ is the risk-aversion parameter. A smaller λ gives a more aggressive portfolio, while a larger λ gives a more conservative portfolio.

The constraints are:

$$\sum_{i=1}^{n} w_i = 1, \qquad 0 \le w_i \le 0.10, \quad i = 1, \ldots, n$$

Thus, the complete optimization problem is:

$$\begin{aligned} \max_{w} \quad & w^T \mu - \lambda w^T \Sigma w \\ \text{s.t.} \quad & \sum_{i=1}^{n} w_i = 1, \\ & 0 \le w_i \le 0.10, \quad i = 1, \ldots, n. \end{aligned}$$

The upper bound of 10% prevents excessive concentration in one asset and forces the portfolio to contain at least 10 assets.

3 Pseudocode for PSO Optimization

Particle Swarm Optimization represents each candidate portfolio as a particle. A particle position is a vector of weights, and its quality is evaluated using the mean-variance objective. After every position update, weights are repaired by clipping them to $[0, 0.10]$ and renormalizing them so that their sum equals one.

Algorithm 1 PSO for Mean-Variance Portfolio Optimization

Require: μ, Σ, λ, particles S, assets n, iterations K, parameters ω, c_1, c_2
Ensure: Best portfolio weights w^*

Randomly initialize positions w_j and velocities v_j, $j = 1, \ldots, S$
Repair each w_j so that $\sum_i w_{j,i} = 1$ and $0 \le w_{j,i} \le 0.10$
Set personal bests $p_j \leftarrow w_j$ and global best $g \leftarrow \arg\max_j f(w_j)$
for $k = 1$ to K **do**
    for $j = 1$ to S **do**
        Generate $r_1, r_2 \sim U(0, 1)$
        $v_j \leftarrow \omega v_j + c_1 r_1 \odot (p_j - w_j) + c_2 r_2 \odot (g - w_j)$
        $w_j \leftarrow w_j + v_j$
        Repair w_j to satisfy $\sum_i w_{j,i} = 1$ and $0 \le w_{j,i} \le 0.10$
        Evaluate $f(w_j) = w_j^T \mu - \lambda w_j^T \Sigma w_j$
        if $f(w_j) > f(p_j)$ **then**
            $p_j \leftarrow w_j$
        end if
        if $f(p_j) > f(g)$ **then**
            $g \leftarrow p_j$
        end if
    end for
end for

For one particle, evaluating $w^T \mu$ costs $O(n)$, while evaluating $w^T \Sigma w$ costs $O(n^2)$ for a dense covariance matrix. Therefore, the cost per particle is $O(n^2)$. For S particles and K iterations, the total PSO time complexity is:

$$O(KSn^2)$$

The memory complexity is:

$$O(n^2 + Sn)$$

because the covariance matrix requires $O(n^2)$ memory and the swarm positions, velocities, and personal bests require $O(Sn)$ memory.

4 Results: Gurobi vs PSO

The optimization was tested with risk-aversion parameter $\lambda = 1.0$.

Table 1: Objective comparison for $\lambda = 1.0$

Solver	Objective	Exp. Return	Variance	Assets	Gap
Gurobi	0.001140	0.001386	0.000246	15	0.000000
PSO	0.001052	0.001247	0.000195	10	0.000088

Gurobi achieved the best objective value, 0.001140, while PSO obtained 0.001052. The gap was:

$$0.001140 - 0.001052 = 0.000088$$

This means that PSO found a feasible solution close to the Gurobi optimum, but did not exactly match it. This is expected because Gurobi solves the quadratic optimization problem directly, while PSO is an iterative metaheuristic.

Gurobi selected 15 assets, while PSO selected 10 assets. Since every asset has a maximum weight of 10%, 10 assets is the minimum possible number needed to fully invest the portfolio. Therefore, the PSO solution was more concentrated, while the Gurobi solution was more diversified. Although PSO produced lower variance, its lower expected return caused a lower final objective value. Overall, Gurobi gives the benchmark optimum, while PSO provides a competitive approximate solution.